МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»

Навчально-науковий інститут комп’ютерних систем

Кафедра інженерії програмного забезпечення

Артем МОРОЗОВ

(група АС-211)

КВАЛІФІКАЦІЙНА РОБОТА БАКАЛАВРА

Інтегрований фреймворк підтримки надійності мікросервісної архітектури

Спеціальність:

121 Інженерія програмного забезпечення

Освітньо-наукова програма:

Інженерія програмного забезпечення

Керівник:

Олексій КУНГУРЦЕВ, к.т.н., професор

Одеса – 2025

АНОТАЦІЯ

ABSTRACT

ЗМІСТ

ВСТУП

У сучасному світі цифровізація торкається практично всіх сфер діяльності людини. Все більше компаній обирають онлайн-модель ведення бізнесу, що потребує надійних та масштабованих програмних рішень. Одним із таких рішень стала мікросервісна архітектура, яка дозволяє розділяти складні системи на окремі, незалежні компоненти. Це дає змогу пришвидшити розробку, тестування та впровадження нових функцій, а також забезпечує гнучкість при масштабуванні.

Проте разом із перевагами мікросервісного підходу виникають і нові виклики. Серед них — складність комунікації між сервісами, необхідність узгодження транзакцій, а також забезпечення стійкості системи до відмов. Особливо актуальними ці виклики стають у критичних галузях, таких як обробка онлайн-замовлень, де збої можуть призвести до втрати клієнтів або фінансових збитків.

У таких умовах постає потреба у впровадженні надійних механізмів обробки подій, автоматичному реагуванні на помилки та забезпеченні спостережуваності за всіма компонентами системи.

Метою кваліфікаційної роботи є зменшення ризику втрати даних і збоїв при обробці онлайн-замовлень шляхом впровадження механізмів надійності у мікросервісну систему

Об’єктом роботи є побудова мікросервісної системи, для обробки онлайн-замовлень, що забезпечує стійкість до збоїв та дозволяє відстежувати всі етапи життєвого циклу замовлення у розподіленому середовищі.

Предметом роботи є програмна реалізація засобів забезпечення надійності мікросервісної взаємодії, зокрема реалізація механізмів Dead Letter Queue (DLQ), спостережуваності за допомогою OpenTelemetry, а також управління транзакціями з використанням Saga-патерну.

Завдання кваліфікаційної роботи:

− проаналізувати існуючі підходи до забезпечення надійності в мікросервісних архітектурах;

− визначити вимоги до функціональності системи електронної комерції;

− спроєктувати архітектуру системи з розподілом на незалежні сервіси;

− реалізувати взаємодію між сервісами

− впровадити механізм обробки помилок

− забезпечити трасування і моніторинг запитів

− протестувати працездатність сервісів

1 СПЕЦИФІКАЦІЯ ВИМОГ ДО СИСТЕМИ

1.1 Актуальність проблеми

Мікросервісна архітектура значно трансформувала підходи до розробки масштабованих програмних систем. За даними опитування CNCF за 2023 рік, понад 78% великих підприємств використовують мікросервіси для своїх критично важливих додатків. Ця архітектура забезпечує незалежне розгортання, оновлення та масштабування окремих компонентів, що значно підвищує гнучкість розробки та експлуатації. Проте, через складність взаємодії між сервісами, ризики відмов значно зростають. Згідно з дослідженням Gartner, компанії втрачають від $300,000 до $1,000,000 на годину через простої критичних систем.

1.2 Опис предменої області

Предметною областю системи є забезпечення надійності мікросервісних архітектур. У сучасних розподілених системах, особливо в сферах електронної комерції, фінансів та логістики, стабільність і узгодженість даних є критично важливими. Високий рівень масштабованості та незалежність сервісів у мікросервісній архітектурі роблять її зручною для розробки великих проєктів, але разом із тим створюють додаткові виклики у сфері транзакцій, комунікації та моніторингу.

У таких системах можуть виникати ситуації, коли:

а) транзакція виконується лише частково, через що стан даних у різних сервісах стає некоректним;

б) повідомлення між сервісами губляться або дублюються;

в) при збої одного сервісу немає механізму швидкого відновлення його роботи;

г) діагностика проблем ускладнюється через великий обсяг логів і відсутність централізованої системи трасування.

Щоб вирішити ці виклики, потрібне комплексне рішення, яке забезпечить не лише узгодженість операцій, а й гарантію доставки повідомлень та ефективний моніторинг системи.

1.3 Аналіз існуючих програмних продуктів

У межах запропонованої системи мікросервісної надійності поставлені проблеми можуть вирішуютися комплексно. Нижче розглянуто 5 альтернативних рішень, які пропонують інтегровані підходи до цих завдань: AWS Step Functions, Temporal.io, Apache Kafka Streams, Datadog APM та New Relic APM. Для кожного рішення описано його функціональність, переваги і недоліки, а також наведено порівняння, у яких випадках варто обрати те чи інше рішення. Для оцінки альтернативних рішень використовуються такі критерії:

а) Гнучкість (адаптація до різних інфраструктур);

б) Масштабованість (підтримка високих навантажень);

в) Інтеграція (сумісність із популярними технологіями);

г) Простота розгортання та підтримки.

AWS Step Functions - це кероване рішення для оркестрації бізнес-процесів у хмарному середовищі AWS. Цей сервіс часто використовується для реалізації патерну Saga (оркестрування) замість двофазного коміту 2PC, щоб забезпечити кінцеву узгодженість у транзакціях, що охоплюють кілька розподілених систем.

Переваги AWS Step Functions:

− Глибока інтеграція з AWS - є безсерверним (serverless) рішенням, інфраструктура автоматично масштабується і керується AWS, що спрощує підтримку

− Автоматичне масштабування - автоматичне керування навантаженням та розподіл ресурсів.

− Мінімум коду - можна налаштувати процеси конфігурацією без необхідності писати складний код.

Так же розглянемо і недоліки:

− Vendor lock-in – працює тільки в екосистемі AWS, що ускладнює міграцію.

− Висока вартість – оплата за кожен перехід між станами

− Обмеженя можливостей - максимальний розмір переданих даних 256 КБ, ліміти на кількість одночасних станів тощо.

Temporal.io - відкрита платформа для оркестрації бізнес-процесів, яка дозволяє реалізовувати розподілені транзакції на рівні коду.

Переваги:

− Підтримка різних мов програмування — пропонує високу гнучкість для розробників, workflows визначаються як код, підтримуються різні мови (Java, Go, Python тощо), що спрощує вираження складної бізнес-логіки

− Можливість розгортання у власному середовищі - можна розгорнути локально або в будь-якому хмарному середовищі, а також існує керований хмарний сервіс Temporal Cloud.

− Гнучке управління станами та таймаутами - усі кроки виконуються із доставкою принаймні один раз з автоматичними ретраями, а фреймворк забезпечує, що кожна задача буде виконана рівно один раз завдяки ідемпотентності та журналу подій.

Недоліки:

− Обмеження транзакційної моделі - вимагає спеціального проектування для забезпечення узгодженості даних

− Залежність від зовнішньої системи - тісна інтеграція створює залежність к Temporal як критичному компоненту. Необхідність моніторингу та забезпечення високої доступності самого Temporal.

− Може бути надлишковим для простих бізнес-процесів

Datadog APM - це хмарна платформа моніторингу та спостереження (observability), що надає комплексні засоби для збору метрик, логів і трасувань від розподілених сервісів.

Переваги:

− Готове рішення для аналізу запитів у реальному часі;

− Інтеграція з OpenTelemetry;

− Єдина платформа для логування, моніторингу та трасування;

− Потужна візуалізація даних і розширена аналітика.

Недоліки:

− Висока вартість при масштабуванні;

− Відсутність локального розгортання (SaaS-модель);

− Можливий vendor lock-in через глибоку інтеграцію з екосистемою Datadog.

В табл 1.1 буде представлений детальний аналіз систем, було визначено та занесено до таблиці основний функціонал, який має містити система для організації заходів. На наступній сторінкі можна побачити таблицю «Порівняння функціоналу аналогів».

Таблиця 1.1 - Порівняння характеристик існуючих рішень

Рішення	Тип	Підтримка масштабування	Гнучкість	Простота інтеграції	Недоліки
AWS Step Functions	Оркестрація Saga	Висока	Середня	Лише для AWS	Працює тільки в AWS (vendor lock-in) , висока вартість, обмеження платформи
Temporal	Оркестрація Saga	Висока	Висока	Гнучка	Вимагає окремої інфраструктури, крива навчання, надлишковість для простих сценаріїв

Datadog APM	Моніторинг	Висока	Висока	Готове рішення	Висока вартість, можливий vendor lock-in

1.4 Функціональні вимоги

Система повинна забезпечувати наступні функціональні можливості:

− Оформлення замовлення - користувач має змогу створити нове замовлення, вказавши перелік товарів та кількість кожного з них.

− Перевірка наявності товарів - система автоматично перевіряє наявність зазначених товарів на складі.

− Резервування товарів - у разі доступності система резервує відповідну кількість товарів для конкретного замовлення.

− Розрахунок загальної вартості - система визначає підсумкову вартість замовлення на основі актуальних цін.

− Ініціація оплати - після підтвердження наявності та вартості, система надсилає запит на оплату.

− Обробка результату оплати - якщо оплата пройшла успішно — замовлення переходить до наступного етапу. У разі неуспішної оплати — виконується скасування замовлення.

− Компенсаційні дії - у разі невдалого виконання одного з етапів система повинна автоматично виконати компенсаційні дії, зокрема скасування резервування товарів та оновлення статусу замовлення.

− Інформування користувача - користувач має отримувати повідомлення про зміну статусу замовлення (успішне оформлення, помилка, скасування тощо).

− Надійність виконання - система має гарантувати узгоджене виконання всіх етапів обробки замовлення — без часткових результатів у разі помилок.

− Протоколювання дій - усі ключові події в обробці замовлення мають реєструватися для подальшого аналізу та відновлення у разі збоїв.

− Модульність та масштабованість - архітектура системи має забезпечувати незалежну роботу кожного функціонального блоку з можливістю масштабування при зростанні навантаження.

1.5 Варіанти використання

Моделювання варіантів використання — це процес опису функціональної поведінки системи з точки зору користувача, який дозволяє виявити основні сценарії взаємодії між користувачем і системою у вигляді варіантів використання (use cases). Такий підхід допомагає зрозуміти, які саме дії має виконувати система для досягнення цілей користувача.

Тому для моделювання взаємодії користувача із системою було розроблено діаграму варіантів використання.

Основним актором є користувач, який взаємодіє із системою через API. Система, у свою чергу, забезпечує необхідні сервіси.

1.6 Сценарії варіантів використання

Варіант використання: «Оформлення замовлення»

Актори: Користувач

Область дії: Система

Передумова: Користувач здійснив вибір товару та бажає його придбати.

Тригер: Користувач натискає кнопку «Оформити замовлення».

Основний сценарій:

1) Користувач обирає товар, вказує бажану кількість товару та підтверджує оформлення замовлення.

2) Система підтверджує наявність товарів у сховищі.

3) Система резервує товари в базі даних.

4) Система надсилає запит на оплату.

5) Створюється запит на доставку.

6) Користувач отримує підтвердження замовлення.

Альтернативний сценарій:

1а. Такого товару не існує

1а1. Система повідомляє користувача про відсутність товару.

2а. Товару немає в наявності.

2а1. Система повідомляє користувача про відсутність товару.

4а. Оплата неуспішна.

4а1. Система скасовує резерв товару.

4а2. Система повідомляє користувача про помилку оплати.

Кінець ВВ.

Варіант використання: Відстеження статусу замовлення

Основний сценарій:

1) Користувач надсилає запит на відстуження замовлення

2) Система запитує у користувача унікальний ідентифікатор замовлення.

3) Користувач вводить ідентифікатор замовлення.

4) Система здійснює пошук замовлення у базі даних.

5) Система повертає користувачеві інформацію про поточний статус замовлення (наприклад: «Замовлення оформлено», «Оплата виконана», «Очікує доставку», «Скасовано»).

Альтернативні сценарії:

2а. Користувач не вводить ідентифікатор замовлення.

2а1. Система повідомляє, що поле обов’язкове для заповнення.

4а. Система не знаходить замовлення з таким ідентифікатором.

4a1. Система повідомляє, що замовлення не існує або було видалено.

Кінець ВВ.

Сценарій 3: Скасування замовлення у разі помилки оплати

Основний сценарій:

1) Користувач оформлює замовлення.

2) Система перевіряє наявність товару та резервує його.

3) Система надсилає запит на оплату.

4) Платіжна система повідомляє про помилку під час обробки платежу.

5) Система скасовує замовлення.

6) Система надсилає команду до сховища для скасування резерву товару.

7) Користувач отримує повідомлення про невдалу оплату та скасування замовлення.

Альтернативні сценарії:

4a. Платіжна система не відповідає.

4a1. Система повторює запит на оплату (до 3 разів).

4a2. Після трьох невдалих спроб — замовлення скасовується.

Кінець ВВ.

1.6 Нефункціональні вимоги

Вимоги до продуктивності:

− Відмовостійкість: система повинна зберігати функціональність при відмові окремих сервісів

− Обробка транзакцій: система має обробляти не менше 1000 транзакцій на секунду під пікових навантаженнях.

− Час відповіді: середній час відповіді для основних операцій не повинен перевищувати 500 мс при стандартному навантаженні.

− Масштабованість: архітектура повинна підтримувати горизонтальне масштабування мікросервісів незалежно один від одного.

Вимоги до надійності:

− Цілісність даних: при збоях в будь-якому мікросервісі система має гарантувати узгодженість даних через механізм Saga для компенсаційних транзакцій.

− Ідемпотентність операцій: усі операції мають бути ідемпотентними для коректної обробки повторних запитів

− Автоматичне відновлення: система повинна автоматично відновлюватися після збоїв, використовуючи механізми Dead Letter Queue та повторні спроби.

Вимоги до моніторингу:

− Трасування запитів: кожен запит повинен мати унікальний ідентифікатор, що дозволяє відстежувати його шлях через всі мікросервіси (distributed tracing).

− Метрики продуктивності: система повинна збирати метрики про час відповіді, кількість помилок, затримки між сервісами.

− Візуалізація даних: усі дані моніторингу мають бути представлені через інтерактивні дашборди Grafana з можливістю налаштування сповіщень.

− Аналіз першопричин: система повинна забезпечувати інструменти для швидкого визначення першопричин збоїв.

Вимоги до розгортання та супроводу:

− Контейнеризація: всі компоненти мають бути упаковані в Docker-контейнери для спрощення розгортання.

Висновок до 1 розділу

У першому розділі було проведено комплексний аналіз архітектури мікросервісної системи, яка реалізує механізми обробки замовлень, платежів, управління складськими залишками, доставки та сповіщень.

Було визначено основні переваги мікросервісного підходу, зокрема масштабованість, гнучкість, надійність та незалежність сервісів. Разом із цим були розглянуті потенційні виклики, такі як забезпечення узгодженості даних між сервісами та обробка помилок у розподіленій системі.

Аналіз аналогів показав, що сучасні розподілені системи використовують

Saga, DLQ та OpenTelemetry для забезпечення надійності та спостережуваності. На основі цього було обрано подієво-орієнтований підхід, що дозволяє ефективно координувати процеси між сервісами.

Були сформульовані функціональні вимоги до системи, які включають розподіл ролей між мікросервісами, використання асинхронних комунікацій та автоматизацію відновлення після збоїв. Також було визначено нефункціональні вимоги, такі як висока доступність, підтримка горизонтального масштабування, безпека даних та моніторинг продуктивності.

Описані сценарії використання дозволили окреслити ключові процеси системи, включаючи оформлення замовлення, обробку платежів, управління складом та повторну спробу доставки через DLQ.

2 ПЛАНУВАННЯ ПРОГРАМНОГО ПРОЄКТУ

2.1 Оцінка тривалості розробки

Успішна реалізація програмного продукту вимагає чіткого та структурованого підходу до планування. У цьому розділі буде проведена оцінка трудовитрат і строків розробки на основі методики Use Case Points (UCP). Цей підхід дозволяє кількісно визначити складність проєкту, оцінити необхідний час виконання та спланувати основні етапи роботи.

Також буде складено графік виконання завдань у вигляді діаграми Ганта, що наочно демонструє послідовність і взаємозалежність процесів. Окремо розглядається аналіз ризиків, який допоможе визначити можливі загрози та способи їхнього мінімізації.

Результати цього розділу дадуть змогу оптимізувати розподіл ресурсів, передбачити можливі затримки та сформувати ефективний план розробки.

Крок 1: Визначемо ваги варіантів використання (UUCW)

Кожен варіант використання (Use Case) класифікується за складністю залежно від кількості взаємодій з акторами та кількості процесів.

Категорії складності Use Case:

- Простий (≤3 взаємодії) → 5 балів
- Середній (4-7 взаємодій) → 10 балів
- Складний (≥8 взаємодій) → 15 балів

Таблиця 2.1 Оцінка складності варіантів використання

Варіант використання	Кількість взаємодій	Категорія	Обґрунтування	Вага
Оформлення замовлення	6	Середній	Залучає Order Service, Inventory Service, Payment Service, Delivery Service. Має перевірку товару, оплату та підтвердження замовлення.	10
Обробка платежу	5	Середній	Прийом запитів на оплату, підтвердження або відмова.	10
Управління складом	4	Середній	Перевірка наявності товару, резервування, зняття резерву у разі скасування. Інтеграція з Order Service та Delivery Service.	10
Обробка доставки	7	Середній	Зміну статусів, скасування доставки у разі відміни замовлення.	10
Сповіщення користувача	3	Простий	Проста логіка: отримати подію → надіслати email/SMS. Мінімальна кількість взаємодій.	5
Повторне виконання через DLQ	8	Складний	Сценарій включає відстеження помилок, повторну відправку повідомлень, логування. Висока складність.	15

Загалом		60

Далі проведемо оцінку технічних факторів. Фактор технічної складності (TCF) використовується для коригування початкової оцінки трудовитрат (UUCP) з урахуванням технологічних аспектів проєкту. Він відображає складність реалізації системи та її архітектурних особливостей. У таблиці наведена оцінка технічних факторів

Таблиця 2.2 – Технічні фактори

Технічний фактор	Вага	Оцінка (0-5)	Обґрунтування	Значення
Дистрибуція системи	2	4	Використовується мікросервісна архітектура, що ускладнює комунікацію між сервісами.	8
Вимоги до продуктивності	2	3	Система повинна підтримувати високу навантаженість, але не є критичною для реального часу.	6
Складність обробки транзакцій	1	4	Підтримка розподілених транзакцій із використанням Saga та компенсаційних механізмів.	4
Частота змін у функціональності	1	3	Проєкт розробляється ітеративно, що потребує частих змін та оновлень.	3
Інші аспекти	1	3	Загальна складність впровадження моніторингу та безпеки.	3
Загалом:				24

Розрахунок TCF:

$$TCF = 0,6 + (0,01 * 24) = 0,84$$

Тепер треба провести оцінку зовнішніх факторів.

Таблиця 2.3 – Зовнішні фактори

Фактор середовища	Вага	Оцінка (0-5)	Значення	Обґрунтування
Досвід роботи з Java та Spring Boot	1.5	4	6	Команда добре знає Java та Spring Boot, що прискорює розробку.
Досвід роботи з мікросервісами	1	3	3	Є базове розуміння мікросервісів, але мало практичного досвіду.
Досвід команди в розподілених системах	0.5	2	1	Недостатній досвід у реалізації масштабованих розподілених систем.
Використання автоматизованого тестування	1	3	3	Планується впровадження тестування, але поки що воно не автоматизоване на 100%.
Чіткість вимог	1	4	4	Є чітке бачення архітектури, що спрощує розробку.
Обмежена кількість часу на роботу над проєктом	1.5	2	3	Розробка ведеться паралельно з іншими завданнями, що зменшує продуктивність.
Загалом			20	

Оцінка зовнішнього фактору:

$$ECF=1.4+(-0.03\times20)=1.4-0.6=0.8$$

Знаючи всі ці дані, можна розрахувати результуючу оцінку:

$$UCP=UUCP\times TCF\times ECF = 60\times0.88\times0.8 = 42.24$$

Щоб отримати загальний час на розробку, використовуємо коефіцієнт продуктивності (Productivity Factor, PF), який визначає, скільки годин розробки потрібно для одного Use Case Point.

Середнє значення PF для мікросервісних систем: PF=20 годин / UCP

Розрахуємо загальну оцінку трудовитрат:

$$T=42.24\times20=844.8 \text{ годин}$$

Оцінений час на розробку вебзастосунку вказує на великий обсяг роботи, оскільки загальний час для виконання всіх етапів перевищує 380 годин для кваліфікаційної роботи. Тому важливо враховувати можливі ризики, а також правильно планувати та ефективно використовувати час.

2.2 Планування робіт проєкту

Визначимо перелік робіт, який нам потрібно виконати. Це дозволить визначити послідовність виконання робіт, контрольні точки (мітки прогресу) та оцінити час на кожен етап.

1. Аналіз — визначаю загальну архітектуру системи, проектую базу даних, розробляю схему взаємодії мікросервісів, визначаю моделі даних.
2. Налаштування інфраструктури - розгортаю локальне середовище розробки, налаштовую систему.
3. Реалізація основних мікросервісів - розробляю основні мікросервіси: Order Service (створення замовлень), Inventory Service (управління складом), Payment Service (обробка платежів), Delivery Service (організація доставки) та Notification Service (сповіщення користувачів).
4. Інтеграція сервісів - забезпечую взаємодію сервісів

5. Тестування та налагодження - пишу юніт-тести та інтеграційні тести

6. Розгортання та підсумкова перевірка - розгортаю систему в хмарному або локальному середовищі

На рисунку представлена діаграма Ганта, яку ми створили для візуалізації плану роботи.

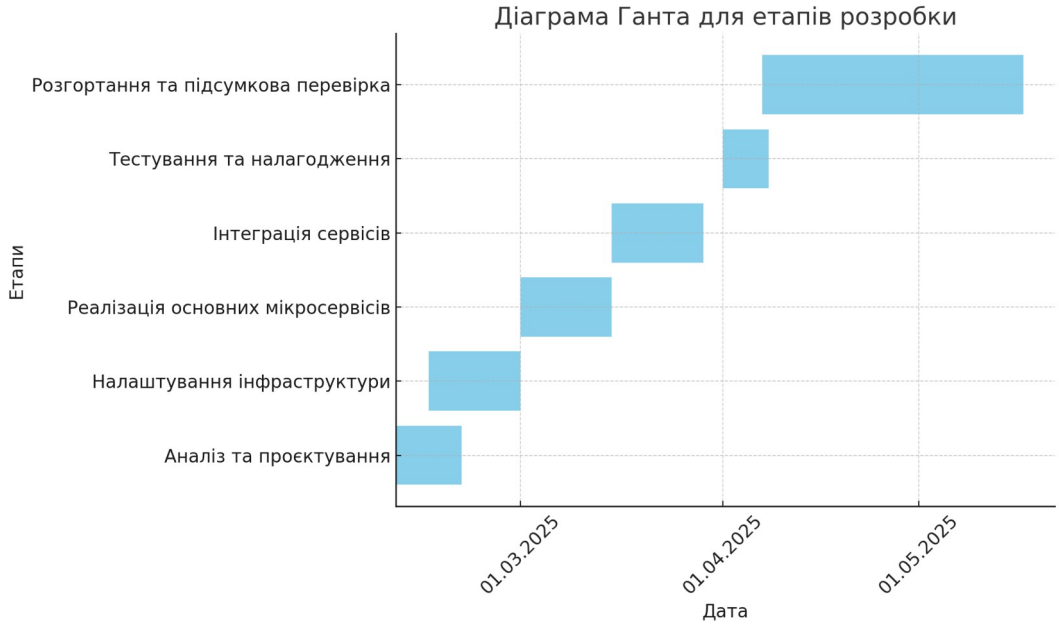

Рисунок 2.4 - діаграма Гантавф

2.3 Аналіз ризиків

Ризик у розробці програмного забезпечення – це ймовірність виникнення подій, що можуть негативно вплинути на терміни, вартість або якість розробки. У рамках нашого проєкту ми можемо виділити кілька основних категорій ризиків: технічні, пов’язані зі складністю реалізації мікросервісної архітектури; організаційні, які стосуються управління процесами та розподілу часу; а також процесні, що можуть впливати на продуктивність та якість кінцевого продукту.

Одним із найбільш критичних ризиків є нестача досвіду роботи з мікросервісами. Хоча команда має добру теоретичну підготовку, розробка реальних розподілених систем з використанням Saga-процесів та подієвої комунікації може викликати труднощі. Це може призвести до архітектурних

помилок, зниження продуктивності та проблем із масштабованістю.

Ще один значний ризик – складність інтеграції сервісів. Використання асинхронної комунікації вимагає ретельного налаштування, щоб уникнути втрати повідомлень і гарантувати узгодженість даних між сервісами. Некоректна інтеграція може призвести до збоїв у роботі системи, а також до складнощів у відстеженні помилок.

Також слід враховувати нестабільність розподілених транзакцій. Використання механізму Saga забезпечує можливість керування транзакціями, однак, якщо компенсаційні операції будуть реалізовані некоректно, це може призвести до неконсистентного стану системи. У випадку збоїв деякі операції можуть бути виконані частково, що спричинить порушення логіки бізнес-процесів.

Окрему увагу варто приділити ризику нестачі часу на реалізацію проєкту. Оскільки це дипломний проєкт, його розробка відбувається паралельно з іншими навчальними завданнями, що може спричинити відставання від графіка. В такому випадку можливе скорочення функціоналу або зниження якості коду через поспіх у реалізації.

Ще одним потенційним викликом є проблеми з тестуванням. Тестування розподіленої системи значно складніше, ніж у випадку монолітної архітектури, оскільки необхідно перевіряти взаємодію сервісів, гарантувати їхню узгодженість та стабільність під навантаженням. Якщо тестування буде недостатньо ретельним, можливі неочікувані збої на продакшені.

Щоб оцінити рівень загрози від кожного ризику, було проведено аналіз на основі ймовірності його виникнення та ступеня впливу. Найбільш критичними факторами є нестача досвіду роботи з мікросервісами (ймовірність – висока, вплив – критичний), а також складність інтеграції сервісів (ймовірність – середня, вплив – високий). Для їхнього зниження будуть застосовані такі методи: вивчення best practices мікросервісної архітектури, використання контрактного тестування та поступове впровадження інтеграції через API Gateway.

Крім того, щоб мінімізувати ризик проблем з транзакціями, буде реалізовано Saga-підхід з компенсаційними операціями та ретельним логуванням стану транзакцій. Для боротьби з нестачею часу передбачено чітке планування задач, використання Agile-методології та автоматизація тестування.

Таким чином, проведений аналіз ризиків дозволяє визначити основні виклики, які можуть вплинути на успішність реалізації проєкту, та розробити стратегії їхньої мінімізації. Використання ефективних підходів до управління ризиками допоможе забезпечити стабільність і якість кінцевого продукту.

Висновки до 2 розділу

У цьому розділі було проведено детальне планування розробки мікросервісної системи, що включало оцінку складності, визначення необхідних ресурсів, переліку робіт та аналіз можливих ризиків.

Для оцінки обсягу робіт використано метод Use Case Points (UCP), який дозволив визначити приблизну трудомісткість проєкту. Враховуючи технічну складність (TCF = 0.88) та середовище розробки (ECF = 0.8), підсумкова оцінка UCP склала 42.24, що відповідає загальному обсягу розробки близько 845 годин.

На основі проведеного аналізу сформовано перелік етапів розробки та створено діаграму Ганта, яка дозволяє візуалізувати часові межі кожного завдання. Основними етапами є: аналіз та проєктування, налаштування інфраструктури, реалізація основних мікросервісів, їхня інтеграція, тестування та розгортання. Також визначено залежності між етапами, що дозволяє ефективно розподілити ресурси.

Було проведено аналіз ризиків, у рамках якого визначено найбільш критичні загрози для проєкту, серед яких нестача досвіду роботи з мікросервісами, складність інтеграції сервісів та нестабільність розподілених транзакцій.

Таким чином, результати цього розділу забезпечують чітке бачення процесу розробки, дозволяють ефективно розподілити час і ресурси, а також передбачити

можливі ризики та способи їхнього усунення. Планування відіграє ключову роль у забезпеченні успішного виконання проєкту та досягнення поставлених цілей.

3 ПРОЕКТУВАННЯ ПРОГРАМНОЇ СИСТЕМИ

3.1 Функціональна декомпозиція системи з виділенням модулівфыв

У цьому розділі буде розглянуто структуру розроблюваної e-commerce системи, побудованої на основі мікросервісної архітектури. Проєктування системи є важливим етапом, оскільки воно визначає, як взаємодіятимуть окремі сервіси, які функції вони виконуватимуть та як забезпечуватиметься узгодженість даних і надійність роботи.

Система складається з --- основних мікросервісів, кожен із яких відповідає за окремий етап обробки замовлення. Використання подієвої моделі комунікації через Kafka, а також механізмів Saga та DLQ дозволяє зменшити ризики втрати даних та гарантувати коректне виконання транзакцій.

У наступних підрозділах буде проведено функціональну декомпозицію системи, побудовано архітектурну модель, розроблено ER-діаграму бази даних, а також розглянуто ключові аспекти роботи Order Service.

Основними модулями системи можна виділити:

1) Order Service - сервіс керування замовленнями. Відповідає за:

− Створення замовлення

− Управління статусами

− Взаємодія з іншими сервісами

2) Inventory Service - сервіс обліку товарів. Відповідає за:

− Перевірка наявності достатньої кількості товару на складах

− Резервування товарів на складах

− Розподілення кількості товару по різних складах при недостачі

− Зняття резерву з товарів на складах

3) Product Service – сервіс управління інформацією про товари. Відповідає за:

− Зберігання актуальної інформації про товари, включаючи ціну;

− Обробку запитів на отримання ціни конкретного товару;

− Повернення відповіді із зазначенням вартості товару на основі даних з бази.

4) Payment Service – сервіс обробки платежів. Відповідає за:

− Обробку платежів на оплату замовлення

− Емуляцію результату платіжної операції

− Надсилання події про успішну або неуспішну оплату

− Передача помилок при оплаті у чергу для подальших повторів та аналізу

5) Delivery Service – сервіс організації доставки. Відповідає за:

− Ініціація доставки замовлення після підтвердження оплати

− Встановлення статусу доставки

− Надсилання події про результат доставки замовлення

6) Notification Service – сервіс повідомлень. Відповідає за:

− Надсилання корисувачу повідомлення про статус замовлення

На рисунку можемо побачити схему взаємодії модулів

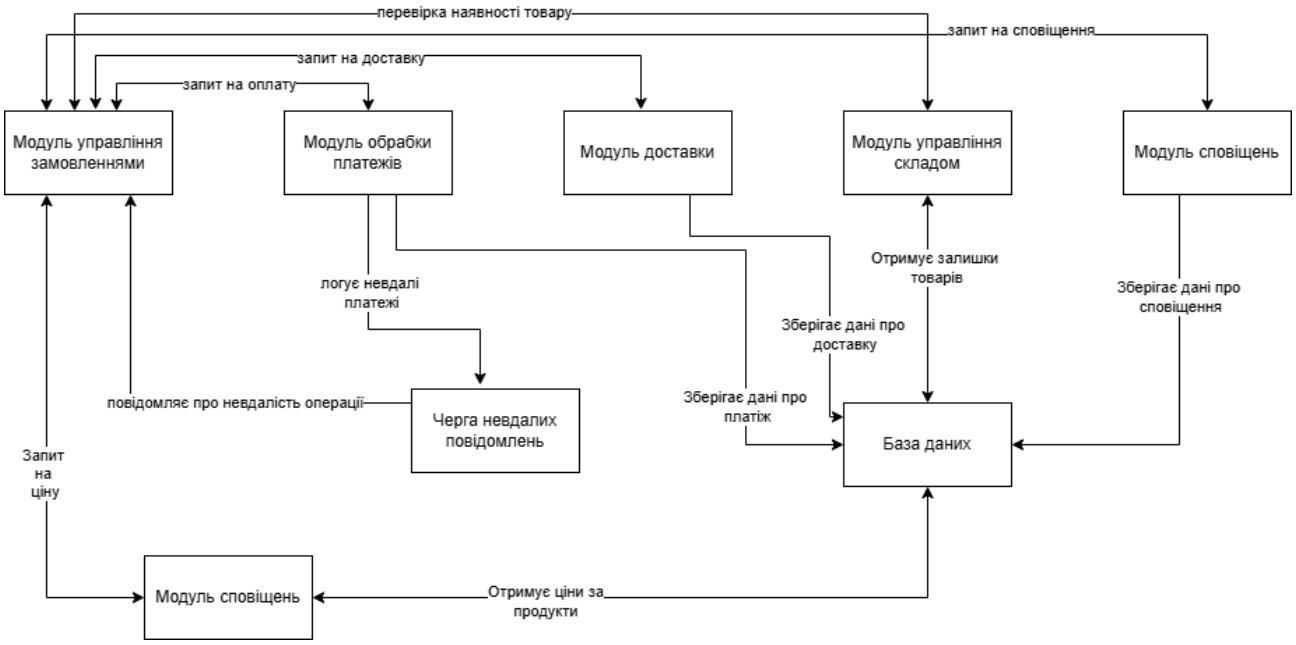

Рисунок — схема взаємодії модулів

3.2 Проектування класів

Концептуальні класи допомагають структурувати програмний код та логіку бота, визначаючи основні сутності та їх взаємодію. Для того, щоб визначити концептуальні класи, потрібно приписати кожньому пункту сценарію відповідний клас.

Табл. 3.1 описує концептуальні класи прецедента «Оформлення замовлення».

Таблиця 3.1 – Концептуальні класи прецедента «Оформлення замовлення»

Зміст пункту основного сценарію (К – користувач, С – система)	Можливий клас
1. Користувач вибирає товари та їх кількість для оформлення замовлення.	User, Product, OrderItem
2. Система перевіряє наявність товару на складі.	Inventory, Product
3. Система резервує товар на складі.	Inventory
4. Система надсилає запит на оплату.	Payment
5. Система після підтвердження оплати надсилає запит на доставку.	Delivery, Order
6. Система інформує користувача про статус замовлення.	Notification, User

Таблиця 3.2 – Концептуальні класи прецедента «Відстеження статусу замовлення»

Зміст пункту основного сценарію	Можливий клас
1. Користувач надсилає запит на перегляд статусу свого замовлення.	User, Order
2. Система отримує ідентифікатор замовлення та перевіряє його статус.	Order
3. Система формує відповідь зі статусом замовлення (наприклад: “Очікує оплату”, “У процесі доставки” тощо).	Order, Notification

Таблиця 3.2 – Концептуальні класи прецедента «Скасування замовлення у разі помилки оплати

Зміст пункту основного сценарію	Можливий клас
1. Користувач оформлює замовлення.	User, Order, Product
2. Система перевіряє наявність товару та резервує його.	Inventory, Product
3. Система надсилає запит на оплату.	Order, Payment
4. Платіжна система повідомляє про помилку під час обробки платежу.	Payment, Notification
5. Система скасовує замовлення.	Order
6. Система надсилає команду до сховища для скасування резерву товару.	Inventory, Product
7. Користувач отримує повідомлення про невдалу оплату та скасування замовлення.	Notification, User
4a. Платіжна система не відповідає. Система повторює запит на оплату (до 3 разів), після чого скасовує замовлення.	Payment, Order, DLQService

Order — основний клас, який представляє замовлення користувача. Містить у собі всю інформацію про товари, стан виконання, дату створення й поточний статус. Є точкою входу в бізнес-логіку системи та ініціатором взаємодії з іншими сервісами.

Product — описує товар, який може бути доданий до замовлення. Містить назву, категорію, виробника, ціну та інші атрибути, які впливають на аналітику, категоризацію та кінцеву ціну.

InventoryItem — представляє товар на складі. Зберігає інформацію про кількість доступних одиниць товару, місце зберігання та підтримує логіку резервування й зняття резерву.

Payment — відповідає за зберігання інформації про платіж, включаючи ідентифікатор замовлення, суму, статус оплати та спосіб оплати. Використовується для ведення обліку платіжних транзакцій і реалізації компенсаційної логіки.

Delivery — описує доставку замовлення. Містить інформацію про адресу доставки, статус, дату надсилання та отримання. Інтегрується з логікою Dead Letter Queue у випадку помилок обробки.

Notification — використовується для надсилання повідомлень користувачу, наприклад, про підтвердження замовлення або помилку оплати. Є допоміжним, але критично важливим компонентом для інформування.

3.2 Розробка ER-моделі системи

Опис зв`язків:

Order → (1 : M) → OrderItem – одне замовлення може містити багато позицій товарів.

OrderItem → (M : 1) → Product – кожна позиція належить одному товару.

Order → (1 : 1) → Payment – кожне замовлення пов’язане з одним платежем.

Order → (1 : 1) → Delivery – кожне замовлення має один запис про доставку.

Inventory → (M : 1) → Product – один товар може бути присутній на

багатьох складах, кожен запис `Inventory` пов’язаний з одним товаром.

Notification → (M : 1) → Order – для одного замовлення може бути надіслано багато сповіщень.

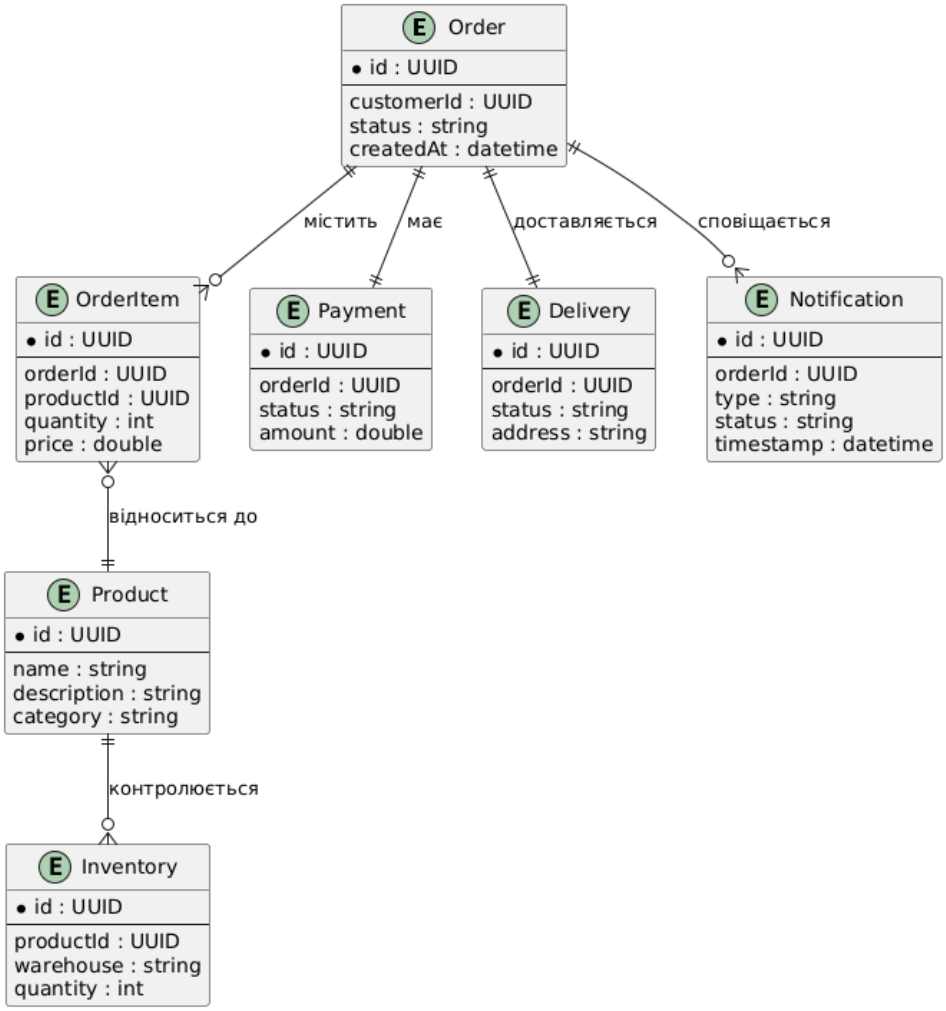

Рисунок - ER-модель системи

3.3 Проектування структур даних

Наступним етапом, спираючись на попередньо побудовану ER модель можна розробити конкретизовану схему базу даних.

Таблиця — опис сутності Order

Сутність	Атрибути	Тип	Опис
Order	id	UUID	Унікальний ідентифікатор повного замовлення

	customer_id	UUID	Ідентифікатор клієнта
	createdAt	LocalDateTime	Дата створення замовлення
	updatedAt	LocalDateTime	Дата оновлення замовлення
	status	ENUM	Статус замовлення (нове, оплачене, доставлене)
	price	Double	Загальна сума замовлення

Таблиця — опис сутності OrderItem

Сутність	Атрибути	Тип	Опис
OrderItem	id	UUID	Унікальний ідентифікатор
	order_id	UUID	Ідентифікатор замовлення (зовнішній ключ)
	productId	UUID	Ідентифікатор продукта (зовнішній ключ)
	quantity	Integer	Кількість
	status	ENUM	Статус
	price	Double	Ціна

Таблиця — опис сутності Product

Сутність	Атрибути	Тип	Опис
Product	id	UUID	Унікальний ідентифікатор товару
	name	String	Назва товару
	description	String	Опис товару
	category	String	Категорія товару

	brand	String	Бренд товару
	supplier	String	Постачальник товару
	price	Double	Ціна за одиницю товару
	currency	ENUM	Валюта для покупки товару
	available	Bool	Доступність товару

Таблиця — опис сутності Inventory

Сутність	Атрибути	Тип	Опис
Inventory	id	UUID	Унікальний ідентифікатор запису
	price	Double	Вартість товару
	product_id	UUID	Ідентифікатор товару (зовнішній ключ)
	warehouseId	Long	Унікальний ідентифікатор окремого склада
	availableQuantity	Bool	Доступна кількість на складі
	quantity	Integer	Кількість товару на складі

Таблиця — опис сутності Payment

Сутність	Атрибути	Тип	Опис
Paymen	payment_id	UUID	Унікальний ідентифікатор платежу

t			
	order_id	UUID	Ідентифікатор замовлення (зовнішній ключ)
	payment_status	ENUM	Статус платежу (успішний, невдалий)

Таблиця — опис сутності Delivery

Сутність	Атрибути	Тип	Опис
Delivery	delivery_id	UUID	Унікальний ідентифікатор доставки
	amount	Double	Сума платежу
	order_id	UUID	Ідентифікатор замовлення (зовнішній ключ)
	status	ENUM	Статус доставки

Таблиця — опис сутності Notification

Сутність	Атрибути	Тип	Опис
Notification	notification_id		Унікальний ідентифікатор повідомлення
	delivery_date		Очікувана дата доставки
	order_id		Ідентифікатор замовлення (зовнішній ключ)
	message		Текст повідомлення
	sent_at		Дата і час відправки повідомлення

Висновки до розділу 3

У цьому розділі було проведено детальне технічне проектування мікросервісної системи, що включає функціональну декомпозицію, моделювання структури даних та взаємодію між компонентами. Це дозволило чітко визначити функціональні області системи та організувати їх ефективну взаємодію.

4 ПРОГРАМНА РЕАЛІЗАЦІЯ СИСТЕМИ

4.1 Вибір інструментів розробки

Для реалізації мікросервісної системи електронної комерції з використанням шаблону транзакцій Saga, асинхронного брокера Kafka та інструментів моніторингу були обрані наступні технології та інструменти.

Java є однією з найпопулярніших мов програмування, яка вирізняється своєю стабільністю, швидкодією та широким набором бібліотек для побудови складних розподілених систем. Використання фреймворку Spring Boot дозволило швидко створити незалежні та легко конфігуровані мікросервіси завдяки вбудованим можливостям автоматичної конфігурації та управління залежностями.

Apache Kafka було обрано як основний брокер повідомлень для забезпечення асинхронного обміну даними між мікросервісами. Kafka відома своєю високою пропускною здатністю, низькою затримкою, масштабованістю та надійністю доставки повідомлень, що ідеально підходить для реалізації шаблону Saga.

Для зберігання даних мікросервісів використано реляційну базу даних PostgreSQL, яка забезпечує надійне та швидке зберігання даних, підтримку транзакцій та хорошу інтеграцію з екосистемою Spring через Spring Data JPA.

Для контейнеризації сервісів і швидкого локального запуску використовувався Docker. Docker Compose дозволив створити єдиний файл конфігурації для запуску всіх необхідних сервісів (Kafka, PostgreSQL,

OpenTelemetry, Prometheus, Grafana, Jaeger), що значно спростило розробку та тестування системи.

Для трасування запитів між мікросервісами та збору метрик використовувалася бібліотека OpenTelemetry, яка забезпечує глибоке розуміння роботи розподілених систем. Jaeger використовувався для зручного аналізу та візуалізації трасувань.

Для збору, зберігання та візуалізації метрик продуктивності та стану системи були обрані Prometheus і Grafana. Ця комбінація дозволила ефективно контролювати роботу сервісів у режимі реального часу, виявляти проблеми на ранніх стадіях та швидко реагувати на інциденти.

Використовувався Maven як інструмент керування збіркою проекту та залежностями. Завдяки модульній структурі Maven забезпечує чітку організацію проекту та просте управління багатомодульними проектами.

Використання цих інструментів дозволило створити надійну, масштабовану і ефективну систему, що відповідає сучасним вимогам до мікросервісних архітектур.

4.2 Аналіз технологій

1) Паттерн Saga

До приходу мікросервісів дані з різних областей зберігалися в одній БД і зміни в цих областях можна було провести атомарно та ізольовано за допомогою транзакцій бази даних. При микросервисной архітектурі дані розподілені відразу з кількох БД, що дозволяє зробити просту транзакцію. Існують способи організації розподілених транзакцій відразу кількох БД, такі як Two-phase commit (2PC). Але такі підходи не набули широкого поширення при використанні мікросервісів у системі через наявність критичних недоліків при суттєвих навантаженнях на систему.

Альтернативою розподіленим транзакціям є організація змін у ключі остаточної узгодженості даних. За такого підходу дані змінюються без ізоляції та

поетапно. Тобто узгодженість забезпечується не одразу на всіх ділянках системи, а поступово. Ця ідея була концептуалізована для мікросервісних систем в архітектурному шаблоні під назвою «сага».

Кожна сага – це набір локальних транзакцій і компенсацій. Локальні транзакції в сазі – це ланцюжок запитів на зміну даних у мікросервісах. У разі виникнення помилки під час застосування локальної транзакції в одному із сервісів система повинна застосувати компенсації у всіх сервісах, які вже виконали локальні транзакції. Застосування компенсацій покликане відновити вихідний стан системи у разі помилок.

У розподіленому програмному рішенні сага може бути реалізована одним із двох способів:

− Хореографія саг – при хореографії сага реалізується в кожному сервісі-учаснику саги. Виконавши свою локальну транзакцію, сервіс надсилає запит наступному в ланцюжку сервісу. Коли всі наступні мікросервіси виконали свою частину змін, вони надсилають підтвердження про успішність виконання попередньому сервісу.

− Оркестрація саг – при оркестрації саг у системі реалізується окремий сервіс («координатор»), відповідальний за управління ходом саг. Це дозволяє здійснити інкапсуляцію логіки, що поєднує різні області застосунку, запобігаючи зміні контрольованих мікросервісів у випадку змін зв’язучої логіки.

У даній системі було обрано підхід оркестрації для координації взаємодії між мікросервісами. Це означає, що логіка виконання складного бізнес-процесу (оформлення замовлення) зосереджена в одному сервісі — order-service, у компоненті OrderSagaManager.

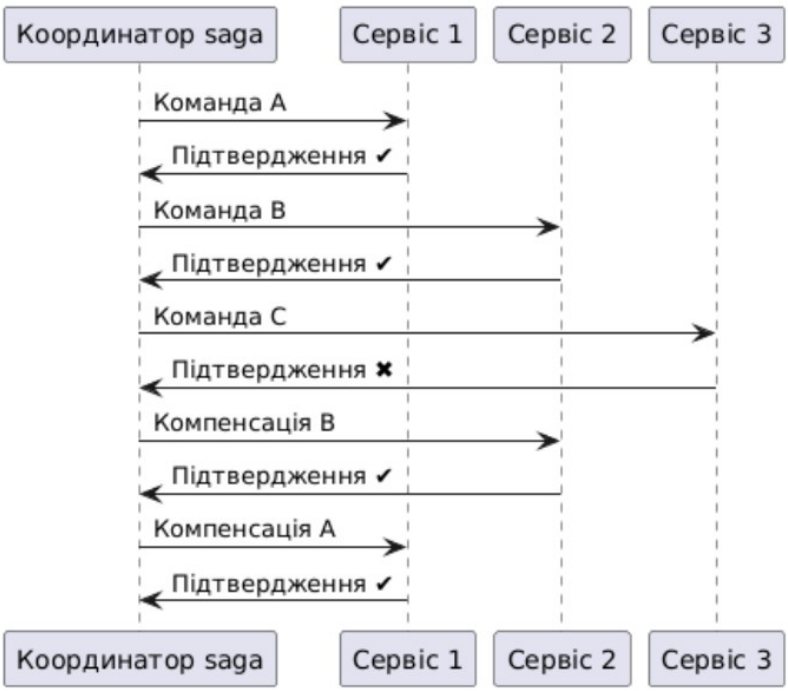

Рисунок — Обробка помилки у локальної транзакції при оркестрації саг

Причини вибору оркестрації:

− Централізований контроль. Усі кроки (резервування товару, запит ціни, оплата тощо) координуються з єдиного місця. Це спрощує розуміння логіки системи.

− Простота впровадження компенсаційних дій. У разі помилки (наприклад, при оплаті) OrderSagaManager викликає відповідні компенсуючі події (наприклад, ReleaseInventoryEvent).

− Послідовність та передбачуваність. Порядок виконання дій чітко регламентований, що знижує ймовірність конфліктів чи непередбачуваної поведінки.

2) Механізм Dead Letter Queue

Dead Letter Queue (DLQ) – це механізм зберігання повідомлень, які не були оброблені або доставлені через певні причини, такі як збій системи, помилки в даних або недоступність отримувача. Можно сказати, що це спеціальна черга повідомлень, в яку надходять невдало оброблені повідомлення.

Призначення DLQ:

− Зберігати повідомлення, які не були оброблені після визначеної кількості спроб.

− Уникнути втрати важливих подій при збоях або непередбачених помилках.

− Дозволити подальший аналіз причин відмов і, за потреби, ручне або автоматичне повторне опрацювання.

У системі DLQ налаштовано для сервісу payment-service,де повідомлення, про оплату, які не вдалося обробити, перенаправляються в топік payment-requests. DLCслухач DLQ (PaymentDlqListener) аналізує повідомлення та автоматично генерує PaymentFailedEvent, що дозволяє order-service виконати компенсаційні дії.

4.3 Опис ключових компонентів системи

4.3.1. Order Service

Мікросервіс order-service є центральним компонентом системи, відповідальним за створення замовлень, координацію транзакцій через паттерн Saga та інтеграцію з іншими сервісами. Нижче описано ключові компоненти сервісу. Основні функції Order Service:

− прийом і збереження нового замовлення;

− генерація подій для резервування товарів;

−очікування та обробка подій InventoryReservedEvent та InventoryNotAvailableEvent;

− ініціація запиту цін на товари (PriceRequestEvent);

− надсилання події оплати (PaymentRequestEvent);

− прийом подій PaymentConfirmedEvent або PaymentFailedEvent;

− виконання компенсаційних дій у разі збоїв.

Сервіс замовлень інтегрується з кількома допоміжними сервісами:

− inventory-service: Для резервування товарів та перевірки їхньої наявності.

− product-service: Для отримання актуальних цін на товари.

− payment-service: Для обробки фінансових транзакцій.

− delivery-service: Для ініціалізація доставки

− notification-service: Для поввідомлення клієнта

Основні компоненти Order Service:

− OrderController — REST-контролер, що приймає запити на створення замовлення.

− OrderSagaManager — координатор Saga, який реалізує кроки обробки замовлення.

− Kafka-лісенери: InventoryEventListener, PriceResponseListener, PaymentEventListener.

−Kafka-продюсери: InventoryEventProducer, PriceRequestProducer, PaymentEventProducer, ReleaseInventoryProducer.

− OrderDbService — обгортка над репозиторієм, яка оновлює статус замовлення, додає ціни тощо.

− TraceService — забезпечує OpenTelemetry-трасування на всіх етапах.

− MetricsService — інтеграція з Micrometer для збору метрик (лічильники, таймери).

REST API Order Service

Сервіс надає доступ до функціональності через REST API. Основні ендпоінти наведено в таблиці 4.3.1.1.

HTTP метод	Маршрут	Опис	Тіло запиту (Request Body)	Тіло відповіді (Response Body)
POST	/api/orders	Створення нового замовлення	OrderRequest	ID створеного замовлення (Long)
GET	/api/orders/{orderId}	Отримання детальної інформації про замовлення	–	OrderResponse

4.3.3. Inventory Service

Мікросервіс inventory-service відповідає за управління складськими запасами та резервування товарів. Він обробляє події OrderCreatedEvent для резервування товарів і ReleaseInventoryEvent для звільнення зарезервованих товарів у разі збою. Розглянемо основні компоненти цього сервісу.

Основні функції Inventory Service:

− зберігання залишків товарів по складах (Inventory);

− підтримка розподіленого резервування (split reservation) — резервування з кількох складів;

− запис окремих резервів у таблицю InventoryReservation;

− скасування резерву при неуспішній оплаті (ReleaseInventoryEvent);

− взаємодія через Kafka з іншими сервісами.

Основні компоненти:

− InventoryService — головний сервіс, який реалізує логіку резервування.

−Kafka-лісенери: OrderCreatedListener (приймає замовлення), ReleaseInventoryListener (обробляє скасування резерву.)

−Kafka-продюсер: InventoryEventProducer (відправляє події з результатами резерву)

Компонент InventoryService реалізує логіку резервування (tryReserve) та звільнення (releaseReservation) товарів. Метод tryReserve розподіляє необхідну кількість товару по складах у порядку зменшення доступної кількості. InventoryEventProducer: Відправляє InventoryReservedEvent або InventoryNotAvailablзалежно від результату резервування. ReleaseInventoryListener: Обробляє ReleaseInventoryEvent для компенсаційних дій. Для замовлення з кількома продуктами order-service генерує набір подій OrderCreatedEvent, кожна з яких ініціює окрему перевірку наявності товару. Ці перевірки можна розглядати як «маленькі» Saga які координуються OrderSagaManager. Тільки після отримання InventoryReservedEvent для всіх продуктів замовлення переходить до наступного етапу (запит цін)

REST API Inventory Service

Сервіс надає доступ до функціональності через REST API. Основні ендпоінти наведено в таблиці.

HTTP метод	Маршрут	Опис	Тіло запиту	Тіло відповіді
GET	/api/inventory/ {productId}	Переглянути залишки певного товару по всіх складах	–	List<InventoryResponse>
GET	/api/inventory/ warehouses/ {warehouseId}	Отримати список товарів та їх кількість на конкретному складі	–	List<InventoryResponse>
GET	/api/inventory/ {productId}/ warehouses	Переглянути, на яких складах зберігається певний товар	–	List<WarehouseInfo>
GET	/api/inventory/ {productId}/ warehouse/ {warehouseId}	Переглянути кількість конкретного товару на конкретному складі	–	InventoryResponse

4.3.3. Product Service

Мікросервіс product-service відповідає за надання цінової інформації про товари, необхідної для формування остаточної вартості замовлення. Сервіс працює на запитах типу PriceRequestEvent і повертає відповідь PriceResponseEvent, що містить ціну конкретного товару.

Основні функції Product Service:

− зберігання даних про товари;

− отримання ціни для кожного товару на запит;

− передача інформації про ціни іншим сервісам через Kafka;

Основні компоненти:

− ProductService — сервіс, що реалізує пошук та перевірку даних про товари.

−Kafka-лісенер: PriceRequestListener (приймає події від Order Service).

− Kafka-продюсер: PriceResponseProducer (відправляє ціну у відповідь).

REST API Product Service

Сервіс надає доступ до функціональності через REST API. Основні ендпоінти наведено в таблиці.

HTTP метод	Маршрут	Опис	Тіло запиту	Тіло відповіді
GET	/api/products	Отримати список усіх товарів	–	List<ProductResponse>
GET	/api/products/{productId}	Отримати детальну інформацію про конкретний товар	–	ProductResponse
GET	/api/products/{productId}/price	Отримати тільки ціну конкретного товару	–	BigDecimal
GET	/api/products/search?name=...	Пошук товарів за частковою назвою або категорією	–	List<ProductResponse>

4.3.4. Payment Service

Мікросервіс payment-service відповідає за обробку платежів і є завершальним етапом у процесі оформлення замовлення. Він інтегрується з order-service через події Kafka та імітує взаємодію з зовнішнім платіжним шлюзом. Розглянемо основні компоненти цього сервісу.

Основні функції Payment Service:

− прийом запиту на оплату;імітація платежу через платіжний шлюз (stub-реалізація);

− надсилання результату оплати;

− обробка подій, що не вдалося обробити — через .DLT;

Основні компоненти:

− Kafka-лісенер: PaymentRequestListener (приймає запити на оплату).

− StubPaymentGateway (імітує випадковий успіх або невдачу оплати).

− PaymentEventProducer (надсилає події про успішну або неуспішну оплату).

−PaymentDlqListener — обробляє повідомлення з топіку .DLT, створює PaymentFailedEvent;

− DeadLetterRecord — окрема подія, яка містить інформацію про помилку, тип, stacktrace.

REST API Payment Service

Сервіс надає доступ до функціональності через REST API. Основні ендпоінти наведено в таблиці.

HTTP метод	Маршрут	Опис	Тіло запиту	Тіло відповіді
GET	/api/ payments/ {orderId}	Отримати інформацію про платіж за замовленням	–	PaymentStatusResponse

4.3.5. Delivery Service

Delivery Service відповідає за моделювання процесу доставки після успішної оплати замовлення. У межах Saga-оркестрації він є фінальним активним учасником, який приймає подію DeliveryRequestEvent та реагує створенням DeliveryScheduledEvent або повідомленням про збій доставки DeliveryFailedEvent.

Основні функції Delivery Service:

− отримання події про необхідність доставки;

− моделювання затвердження доставки;

− генерація подій про успішну або неуспішну доставку;

Основні компоненти:

− Kafka-лісенер: DeliveryRequestListener (приймає DeliveryRequestEvent).

− DeliveryService — логіка призначення доставки.

−DeliveryEventProducer — надсилає DeliveryScheduledEvent або DeliveryFailedEvent.

REST API Delivery Service

Сервіс надає доступ до функціональності через REST API. Основні ендпоінти наведено в таблиці.

HTTP метод	Маршрут	Опис	Тіло запиту	Тіло відповіді
GET	/api/ deliveries/ {orderId}	Перегляд статусу доставки за замовленням	–	DeliveryStatusResponse

4.3.6. Notification Service

Notification Service є допоміжним сервісом, який виконує функцію сповіщення користувачів або адміністрації про ключові події в системі. Він реагує на фінальні події життєвого циклу замовлення: підтвердження оплати, доставку або скасування. Цей сервіс демонструє принцип асинхронного реагування на події без зворотного впливу на основну бізнес-логіку.

Основні функції Notification Service:

− слухати Kafka-події: PaymentConfirmedEvent, PaymentFailedEvent, DeliveryScheduledEvent, DeliveryFailedEvent, OrderCancelledEvent;

− формувати текст сповіщення;

− емулювати надсилання (через лог, email, console);

Основні компоненти:

− Kafka-лісенер: NotificationEventListener (обробляє події різних типів).

− NotificationService — логіка формування та "відправки" повідомлень;

− NotificationLogRepository — зберігає історію повідомлень;

REST API Notification Service

Сервіс надає доступ до функціональності через REST API. Основні ендпоінти наведено в таблиці.

HTTP метод	Маршрут	Опис	Тіло запиту	Тіло відповіді
GET	/api/notifications/{orderId}	Отримати всі повідомлення, пов’язані із замовленням	–	List<NotificationResponse>
POST	/api/notifications/send/test	Створити тестове повідомлення вручну	JSON	NotificationResponse

Висновки до розділу 4

Сервіс надає доступ до функціональності через REST API. Основні ендпоінти наведено в таблиці.У межах четвертого розділу було здійснено реалізацію мікросервісної архітектури системи обробки замовлень, орієнтованої на асинхронну взаємодію компонентів за допомогою Apache Kafka. Основну увагу приділено побудові й узгодженій роботі таких сервісів, як Order Service, Inventory Service, Product Service, Payment Service, Delivery Service та Notification Service.

Кожен сервіс реалізує чітко визначену бізнес-функцію та взаємодіє з іншими компонентами виключно через події, що дозволяє забезпечити слабке

зв’язування, масштабованість і гнучкість системи. Особливу роль у забезпеченні узгодженості розподілених транзакцій відіграє оркестраційний патерн Saga, який реалізовано на стороні Order Service.